

CardiolA – IR Além 1

Extração de Informações Clínicas com IA Generativa

Projeto: CardiolA – Fase 5, Capítulo 1

Aluno: Thiago Paraizo da Silva – RM566159 | 2TIAOA-2026

Este notebook usa a API da **DeepSeek** (`deepseek-chat`) para extrair, a partir de relatos clínicos em texto livre, uma estrutura JSON padronizada (dados do paciente, sintomas, histórico, sinais vitais e nível de urgência sugerido).

A DeepSeek oferece uma API compatível com o padrão OpenAI — basta trocar a `base_url` e o nome do modelo. Obtenha sua chave gratuita em: [🔑 https://platform.deepseek.com/api_keys](https://platform.deepseek.com/api_keys)

⚠ Os relatos abaixo são **fictícios**, criados apenas para demonstração.

```
# Célula 1 – Instalar dependência
# O SDK da OpenAI funciona com a DeepSeek sem modificações adicionais
!pip install -q openai
```

```
# Célula 2 – Setup DeepSeek (Corrigido para Colab)
import json
import os
from openai import OpenAI

# — NOVO: Carregar Secret do Colab —————
# No Colab, use o ícone da chave 🔑 na barra lateral para criar
# um Secret com o nome "DEEPSEEK_API_KEY".
try:
    from google.colab import userdata
    os.environ["DEEPSEEK_API_KEY"] = userdata.get('DEEPSEEK_API_KEY')
except ImportError:
    # Caso não esteja no Colab, assume variável de ambiente local
    pass

DEEPSEEK_API_KEY = os.environ.get("DEEPSEEK_API_KEY")

if not DEEPSEEK_API_KEY or DEEPSEEK_API_KEY == "cole_sua_chave_aqui":
    raise ValueError("❌ DEEPSEEK_API_KEY não configurada corretamente!")

client = OpenAI(
    api_key=DEEPSEEK_API_KEY,
    base_url="https://api.deepseek.com"
)

MODEL = "deepseek-chat"
print(f"✅ Cliente DeepSeek configurado | modelo: {MODEL}")
```

✅ Cliente DeepSeek configurado | modelo: deepseek-chat

```
# Célula 3 – Textos clínicos simulados de entrada
# (relatos fictícios, usados apenas para demonstrar a extração)
relatos = [
    "Paciente de 58 anos, sexo masculino, relata dor no peito há 2 dias, "
    "com irradiação para o braço esquerdo. Fumante, hipertenso. "
    "Pressão medida hoje: 165/95 mmHg. Frequência cardíaca: 102 bpm.",

    "Mulher, 42 anos. Queixa de palpitações frequentes, principalmente à noite. "
    "Histórico familiar de arritmia. Não usa medicamentos. "
    "Sente tontura ocasional. Glicemia em jejum: 98 mg/dL.",

    "Homem, 70 anos, diabético e sedentário. Relata falta de ar ao subir escadas, "
    "piora nas últimas 3 semanas. Edema nos tornozelos. "
    "Em uso de Losartana 50mg e Metformina 850mg.",
]

print(f"📄 {len(relatos)} relatos clínicos carregados.")
```

📄 3 relatos clínicos carregados.

```
# Célula 4 – Prompt do sistema e função de extração
SYSTEM_PROMPT = """
Você é um sistema de extração de informações clínicas.
Analise o relato e retorne APENAS um JSON válido com a estrutura abaixo.
Não adicione explicações, markdown ou texto fora do JSON.
Se um campo não estiver mencionado no relato, use null ou lista vazia.
```

```
{
  "paciente": {
    "idade": null,
    "sexo": null
  },
  "sintomas_principais": [],
  "historico": {
    "comorbidades": [],
    "medicamentos_em_uso": [],
    "historico_familiar": []
  },
  "sinais_vitais": {
    "pressao_arterial_mmHg": null,
    "frequencia_cardiaca_bpm": null,
    "glicemia_mg_dl": null
  },
  "nivel_urgencia": "baixo|medio|alto|emergencia",
  "recomendacao_triagem": ""
}
"""

def extrair_informacoes_clinicas(relato: str) -> dict:
    """Envia o relato para a DeepSeek e retorna um dict com as informações extraídas."""
    response = client.chat.completions.create(
        model=MODEL,
        response_format={"type": "json_object"}, # garante JSON válido na saída
        messages=[
            {"role": "system", "content": SYSTEM_PROMPT},
            {"role": "user", "content": f"Relato clínico:\n{relato}"},
        ],
        temperature=0.1, # baixo para maior determinismo – importante em contexto clínico
    )
    return json.loads(response.choices[0].message.content)

print("✅ Função de extração pronta.")
```

✅ Função de extração pronta.

```
# Célula 5 – Processar e exibir cada relato
resultados = []

for i, relato in enumerate(relatos, 1):
    print(f"\n{'=' * 55}")
    print(f"RELATO {i}:")
    print(relato)
    print("\n 📄 EXTRAÇÃO ESTRUTURADA (DeepSeek):")

    resultado = extrair_informacoes_clinicas(relato)
    resultados.append(resultado)
    print(json.dumps(resultado, ensure_ascii=False, indent=2))

# Destaca o nível de urgência
nivel = resultado.get("nivel_urgencia", "?")
icone = {"baixo": "🟢", "medio": "🟡", "alto": "🟠", "emergencia": "🔴"}.get(nivel, "🟣")
print(f"\n{icone} Urgência: {nivel.upper()}")
```

```
=====
RELATO 1:
Paciente de 58 anos, sexo masculino, relata dor no peito há 2 dias, com irradiação para o braço esquerdo. Fumante, hiperte

📄 EXTRAÇÃO ESTRUTURADA (DeepSeek):
{
  "paciente": {
    "idade": 58,
    "sexo": "masculino"
  },
  "sintomas_principais": [
    "dor no peito",
    "irradiação para o braço esquerdo"
  ],
  "historico": {
    "comorbidades": [
      "hipertensão"
    ],
    "medicamentos_em_uso": [],
    "historico_familiar": []
  },
  "sinais_vitais": {
    "pressao_arterial_mmHg": "165/95",
    "frequencia_cardiaca_bpm": 102,
    "glicemia_mg_dl": null
  }
}
```

```

    },
    "nivel_urgencia": "alto",
    "recomendacao_triagem": "Encaminhamento imediato para avaliação cardiológica de emergência devido a dor torácica com irr
  }

```

● Urgência: ALTO

=====

RELATO 2:

Mulher, 42 anos. Queixa de palpitações frequentes, principalmente à noite. Histórico familiar de arritmia. Não usa medicam

EXTRAÇÃO ESTRUTURADA (DeepSeek):

```

{
  "paciente": {
    "idade": 42,
    "sexo": "feminino"
  },
  "sintomas_principais": [
    "palpitações frequentes",
    "tontura ocasional"
  ],
  "historico": {
    "comorbidades": [],
    "medicamentos_em_uso": [],
    "historico_familiar": [
      "arritmia"
    ]
  },
  "sinais_vitais": {
    "pressao_arterial_mmHg": null,
    "frequencia_cardiaca_bpm": null,
    "temperatura": null,
    "frequencia_respiratoria": null,
    "saturacao_oxigenio": null
  }
}

```

```

# Célula 6 – Salvar resultados em JSON
output_path = "resultados_extracao.json"

with open(output_path, "w", encoding="utf-8") as f:
    json.dump(resultados, f, ensure_ascii=False, indent=2)

print(f"✅ Resultados salvos em '{output_path}'")

```

✅ Resultados salvos em 'resultados_extracao.json'

Discussão

Por que DeepSeek?

A DeepSeek oferece uma API **compatível com o padrão OpenAI** (mesmos parâmetros, mesmo SDK), com custo muito menor e plano gratuito generoso para prototipagem. A troca em relação a GPT-4o se resume a duas linhas: `base_url` e nome do modelo.

`response_format: json_object`

Obriga o modelo a devolver um JSON sintaticamente válido, eliminando a etapa frágil de parsear texto livre com regex.

Temperatura baixa (0.1)

Reduz a variação entre execuções — importante em um contexto clínico, onde a extração deve ser o mais determinística possível.

Limitações e considerações éticas

- O modelo pode **alucinar valores** quando o relato é ambíguo. O prompt instrui que campos ausentes devem ser `null` — não inferidos.
- `nivel_urgencia` e `recomendacao_triagem` são **sugestões de apoio à triagem**, não um diagnóstico médico. A decisão final é sempre de um profissional de saúde.
- Dados clínicos são sensíveis (LGPD / HIPAA): em produção, o relato não deve ser enviado a uma API externa sem **anonimização** e sem base legal adequada.

